# Two quick exams, then: how to beat Friday

TC1037 · Algorithmic Thinking · Session 9 · midterm review

Dr. Leopoldo Rodríguez Salazar

Tuesday 20 October 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

# Two minutes, two requests

## INE — credencial y voto informado

FETEC's one-minute survey: who already has an INE and who needs one for the 2027 elections.

`forms.cloud.microsoft/r/ZrqcPUndjT`

Do it now, on your phone.

## Levelling-module survey (IFE)

12 minutes, personal link in your Tec email — search *Instituto para el Futuro de la Educación* (check spam).

**Closes Friday 23 October.**

# Friday 23 October — the midterm, on paper

**Part I — read and predict · 30 pts**

Six short programs: arithmetic · boolean · conditional · functions · `while` · `for`.
Write *exactly* what Python prints — every line, in order.
5 points each, all or nothing.

**Part II — write the program · 70 pts**

| | |
|---|---|
| a function with a `while` | 23 |
| a function with a `for` | 23 |
| `menu()` | 3 |
| `main()`: loop, validate, call, print | 20 |
| last line calls `main()` | 1 |

80 minutes · no computer, phone, notes or calculator · pen · versions A and B

**Two practice papers with solutions — on the site now** (Exams page).

# Part I — the only technique: a table, one row per turn

```
n = 50
count = 0
while n > 5:
    n = n // 2
    count = count + 1
print(n, count)
```

| **turn** | `n > 5?` | `n` | `count` |
|---|---|---|---|
| start | — | 50 | 0 |
| 1 | yes | 25 | 1 |
| 2 | yes | 12 | 2 |
| 3 | yes | 6 | 3 |
| 4 | yes | 3 | 4 |
| 5 | **no** | | |

# Part I — the only technique: a table, one row per turn

```
n = 50
count = 0
while n > 5:
    n = n // 2
    count = count + 1
print(n, count)
```

| **turn** | `n > 5?` | `n` | `count` |
|---|---|---|---|
| start | — | 50 | 0 |
| 1 | yes | 25 | 1 |
| 2 | yes | 12 | 2 |
| 3 | yes | 6 | 3 |
| 4 | yes | 3 | 4 |
| 5 | **no** | | |

**Output:** `3 4`

Practice 1, question 5. The test is checked *before* every turn. The variables after the last "yes" row are what gets printed.

# Part I — three traps that are always there

## Precedence

```
print(10 - 6 / 3 * 2 +
    15 // 4)
```

# Part I — three traps that are always there

## Precedence

```
print(10 - 6 / 3 * 2 +
    15 // 4)
```

6/3*2 = 4.0 (float!) · 15//4 = 3 → 9.0

** before * / // % before + -. One / makes the whole result a float.

## Boolean order

```
print(4 > 3 and not 2
    == 2.0
      or 7 != 7)
```

# Part I — three traps that are always there

## Precedence

```
print(10 - 6 / 3 * 2 +
    15 // 4)
```

6/3*2 = 4.0 (float!) · 15//4 = 3 → 9.0

** before * / // % before + -. One / makes the whole result a float.

## Boolean order

```
print(4 > 3 and not 2
    == 2.0
      or 7 != 7)
```

not → and → or. 2 == 2.0 is True, so not gives False → False and ... → False or False → False

## Scope

```
def h(x, y):
    x = x + y
    y = x - y
    return x * y

def main():
    x = 2
    y = 3
    z = h(y, x)
    print(x, y, z)
main()
```

# Part I — three traps that are always there

## Precedence

```
print(10 - 6 / 3 * 2 +
    15 // 4)
```

6/3*2 = 4.0 (float!) · 15//4 = 3 → 9.0

** before * / // % before + -. One / makes the whole result a float.

## Boolean order

```
print(4 > 3 and not 2
    == 2.0
      or 7 != 7)
```

not → and → or. 2 == 2.0 is True, so not gives False → False and ... → False or False → False

## Scope

```
def h(x, y):
    x = x + y
    y = x - y
    return x * y

def main():
    x = 2
    y = 3
    z = h(y, x)
    print(x, y, z)
main()
```

h(3, 2): x=5, y=3 → 15. main's x, y untouched → 2 3 15

# Part II — write it in this order

1. **The two functions first** — 46 of the 70 points. Header, accumulator, loop, `return`. No `input`, no `print`.
2. **Test them in your head** with the examples on the paper — *especially the smallest one* (`0`, `1`).
3. `menu()` — three `print`s. Three points; do not spend time on it.
4. `main()` — the skeleton you have written in every lab since week 4 →

```
def main():
    keep_going = True
    while keep_going:
        menu()
        option = int(input("Option: "))
        if option == 1:
            ...  # 4 steps
        elif option == 2:
            ...  # same
        elif option == 3:
            print("Goodbye.")
            keep_going = False
        else:
            print("Invalid option.")
            keep_going = False  # ends
main()
```

# Practice 1, live: `sum_digits(n)` with `while`

```
# sum of the digits of n
def sum_digits(n):
    total = 0
    while n > 0:
        total = total + n % 10
        n = n // 10
        # n loses a digit
    return total
```

`n % 10` is the last digit; `n // 10` drops it.

Check the examples: 1234 → 10, 1000 → 1, 0 → 0.

**Ask yourself every time:** what does my loop do when the input is the smallest legal value? Here, 0: zero turns, returns 0 — right. *On Friday it may not be.*

# Practice 1, live: `count_multiples(n, k)` with `for` — and `main`'s option 2

```python
# multiples of k in 1..n
def count_multiples(n, k):
    count = 0
    for i in range(1, n + 1):
        if i % k == 0:
            count = count + 1
    return count
```

```python
elif option == 2:
    n = int(input("n (1 or more): "))
    k = int(input("k (1 or more): "))
    if n < 1 or k < 1:
        print("n and k must be 1 or more.")
    else:
        print("Multiples:",
              count_multiples(n, k))
```

Read · validate · call · print *with a label*. Four lines of thought, every option.

`range(1, n + 1)`: the `+ 1` is written every time, forever. `count_multiples(12, 3)` → 4, not 3.

# How Part II is graded

| | |
|---|---|
| **Excelente 100%** | no syntax errors; works, *including the smallest examples* |
| **Muy bien 85%** | right approach; one small slip (the `+ 1`, the `0` case) |
| **Regular 70%** | at most two syntax errors and/or small logic errors; clearly the right idea |
| **Deficiente 50%** | loosely related, or more than two syntax errors |
| **Muy deficiente 20%** | code unrelated to the problem |
| **Sin evidencia 0** | nothing written |

Per part: $23 \cdot 23 \cdot 3 \cdot 20 \cdot 1$. Then $-2$ for each missing: meaningful names · a prompt for every input · labelled output.

**On paper, syntax still counts:** colons, indentation, parentheses. Write it as if Thonny were reading it.

## Before Friday

1. **Practice midterms 1 and 2** — site → Exams. Solve on paper, 80 minutes, *then* open the solutions.
2. **Friday 16:30 Lab 5** with Luis — the academia's *for y while* lab 2 (primes, perfect numbers). Then **17:35 the midterm**, Room 4206. Bring a pen.
3. **Homework 5** — Sunday 25 October, 23:59.
4. QE3: today's sitting is your QE3 grade; the 13 Oct attempt can only replace a lower quick exam.
5. **Situación problema:** join your team in Canvas (People → Groups) if you have not. Dataset: Friday 6 November.

Next week is Tec Week — no classes. We come back Tuesday 3 November with lists.

# The smallest input first.

Then the `+ 1`. Then `main`.